

Translated

From: e-maxx.ru

Suffix Tree. Ukkonen's Algorithm

This article is a stub and doesn't contain any descriptions. For a description of the algorithm, refer to other sources, such as [Algorithms on Strings, Trees, and Sequences](#) by Dan Gusfield.

This algorithm builds a suffix tree for a given string s of length n in $O(n \log(k))$ time, where k is the size of the alphabet (if k is considered to be a constant, the asymptotic behavior is linear).

The input to the algorithm are the string s and its length n , which are passed as global variables.

The main function `build_tree` builds a suffix tree. It is stored as an array of structures `node`, where `node[0]` is the root of the tree.

In order to simplify the code, the edges are stored in the same structures: for each vertex its structure `node` stores the information about the edge between it and its parent. Overall each `node` stores the following information:

- `(l, r)` - left and right boundaries of the substring `s[l..r-1]` which correspond to the edge to this node,
- `par` - the parent node,
- `link` - the suffix link,
- `next` - the list of edges going out from this node.

```
string s;
int n;

struct node {
    int l, r, par, link;
    map<char, int> next;

    node (int l=0, int r=0, int par=-1)
        : l(l), r(r), par(par), link(-1) {}
    int len() { return r - l; }
    int &get (char c) {
        if (!next.count(c)) next[c] = -1;
        return next[c];
    }
};
```